

```
// PROVIDER LAYER. The rest of JARVIS
only ever calls
provider.chat(messages).
// messages = [{ role: "system" |
"user" | "assistant", content: "text"
}, ...]
// To add Claude, ChatGPT or anything
else later: write one more small
class here.

import { JarvisError } from
"./errors.js";

const defaultFetch = (...args) =>
globalThis.fetch(...args);

function classifyHttp(status) {
  if (status === 401 || status ===
403) return "auth";
  if (status === 429) return "quota";
  if (status >= 500) return
"unavailable";
  if (status === 400 || status ===
404) return "bad_request";
  return "bad_response";
}
```

```
async function postJson(url, headers,
body, { fetchImpl, signal }) {
  let res;
  try {
    res = await fetchImpl(url, {
      method: "POST",
      headers: { "Content-Type":
"application/json", ...headers },
      body: JSON.stringify(body),
      signal,
    });
  } catch (e) {
    if (e && e.name === "AbortError")
throw new JarvisError("aborted", "",
e);
    throw new JarvisError("network",
String((e && e.message) || e), e);
  }
  if (!res.ok) {
    let detail = "";
    try { detail = (await
res.text()).slice(0, 500); } catch {
/* ignore */ }
    throw new
JarvisError(classifyHttp(res.status),
`HTTP ${res.status}: ${detail}`);
  }
  try {
```

```

        return await res.json();
    } catch (e) {
        throw new
JarvisError("bad_response", "Reply
was not valid JSON", e);
    }
}

function needText(text) {
    if (typeof text !== "string" ||
!text.trim()) throw new
JarvisError("bad_response", "Empty
reply");
    return text.trim();
}

// Works with Groq, OpenAI,
OpenRouter and most other "OpenAI-
compatible" services.
export class OpenAICompatibleProvider
{
    constructor({ apiKey, model,
baseUrl, fetchImpl = defaultFetch })
{
        Object.assign(this, { apiKey,
model, baseUrl, fetchImpl });
    }

    async chat(messages, { signal } =

```

```

    {})) {
        if (!this.apiKey) throw new
        JarvisError("no_key");
        if (!this.model || !this.baseUrl)
        throw new
        JarvisError("bad_settings");
        const data = await postJson(
            `${this.baseUrl.replace(/\/+$/,
            "")}/chat/completions`,
            { Authorization: `Bearer
            ${this.apiKey}` },
            { model: this.model, messages
            },
            { fetchImpl: this.fetchImpl,
            signal },
            );
        return needText(data &&
        data.choices && data.choices[0] &&
        data.choices[0].message &&
        data.choices[0].message.content);
    }
}

export class GeminiProvider {
    constructor({ apiKey, model,
    baseUrl, fetchImpl = defaultFetch })
    {
        Object.assign(this, { apiKey,
        model, baseUrl, fetchImpl });
    }
}

```

```

    }

    async chat(messages, { signal } =
    {}) {
        if (!this.apiKey) throw new
        JarvisError("no_key");
        if (!this.model || !this.baseUrl)
        throw new
        JarvisError("bad_settings");
        const system =
        messages.filter((m) => m.role ===
        "system").map((m) =>
        m.content).join("\n");
        const contents = messages
        .filter((m) => m.role !==
        "system")
        .map((m) => ({ role: m.role ===
        "assistant" ? "model" : "user",
        parts: [{ text: m.content }] }));
        const body = { contents };
        if (system)
        body.systemInstruction = { parts: [{
        text: system }] };
        const data = await postJson(
        `${this.baseUrl.replace(/\/+$/,
        "")}/models/${encodeURIComponent(this
        .model)}:generateContent`,
        { "x-goog-api-key": this.apiKey
        },

```

```

        body,
        { fetchImpl: this.fetchImpl,
signal },
    );
    const parts = data &&
data.candidates && data.candidates[0]
&& data.candidates[0].content &&
data.candidates[0].content.parts;
    return
needText(Array.isArray(parts) ?
parts.map((p) => p.text ||
"").join("") : "");
    }
}

// cfg comes from
config.getProviderSettings()
export function createProvider(cfg,
fetchImpl = defaultFetch) {
    if (cfg.type === "gemini") return
new GeminiProvider({ ...cfg,
fetchImpl });
    return new
OpenAICompatibleProvider({ ...cfg,
fetchImpl });
}

```